

ASSESSMENT DECLARATION COVER SHEET

Please complete this cover sheet for each assessment submission. For group assessments, each group member must individually complete and submit a cover sheet.

STUDENT (REGISTRATION) NUMBER: 40735842

MODULE TITLE: SET09120 2025-6 TR1 001 - Data Analytics

DATE OF SUBMISSION: 12/12/2025

I declare that, except where explicitly acknowledged*, this assessment is my own work and has not been submitted for any other module or degree programme at Edinburgh Napier University or any other institution. This declaration is made in compliance with Edinburgh Napier University's [Academic Integrity Regulations](#).

***IMPORTANT:** Contributions from other sources include incorrectly cited quotations or content generated by Generative Artificial Intelligence (Gen AI) tools, such as ChatGPT. See [Artificial Intelligence Tools and Your Learning](#) for more information.

Note: Unless stated in the Assessment Brief, use of Grammarly for checking spelling and grammar is allowable in this assessment, but it cannot be used to generate content.

Please declare here your use of such tools in this assessment submission:
(Select one of the two options below)

☐ Yes, I have used Gen AI tools for this submission and I have described how below.

☒ No, I have not used Gen AI tools for this submission.

If you selected **Yes**, briefly describe (in less than 100 words) how you used these tools:

Click or tap here to enter text.

1.Introduction

In this coursework, I used Weka to analyse the classic German credit dataset and address a practical question: **who can we reliably consider safe to lend money to?**

The dataset contains 1,000 loan applicants, each described by attributes such as checking account status, credit history, loan purpose, savings, personal status, age, and job type, along with a final class label indicating whether the customer ultimately turned out to be a **good** or **bad** credit risk. The class distribution is imbalanced, with 700 good and 300 bad cases.

The main aim of CW2 is to go beyond the basic data cleaning from CW1 and actually carry out **data mining**:

- Discover patterns that separate low-risk applicants from high-risk applicants.
- Investigate whether certain groups (such as skilled workers, older clients, or customers with particular credit histories) have a clear advantage.
- Evaluate a range of algorithms and decide which ones offer the most practical value for a bank.

All experiments were carried out in **Weka**. In line with the coursework brief, I used three different categories of algorithms and matched each one to a suitable version of the dataset:

- **Classification** – J48 decision tree on a mixed numeric/nominal dataset.
- **Association rules** – Apriori on a fully nominal dataset.
- **Clustering** – SimpleKMeans on a small numeric subset.

Together, these three methods offer complementary perspectives on the same data: classification emphasises prediction, association rules highlight strong “IF ... THEN ...” relationships, and clustering reveals natural groupings of applicants.

2. Data preparation and cleaning

2.1 Starting point and fixing CW1 issues

For CW2, I began with the same resources I used in CW1: the cleaned CSV file `SET09120_CW1_40735842_CLEANING.csv` and the Python notebook `40735842_Cw1_pf.ipynb`, which I had used for the first round of cleaning.

My CW1 feedback, however, pointed out that the data was still **not properly cleaned**. So for CW2 I went back, reviewed everything more carefully, and carried out the final preparation in Weka. The final CSV file has 1,000 rows and includes the following attributes: `case_no`, `checking_status`, `credit_history`, `purpose`, `credit_amount`, `saving_status`, `personal_status`, `age`, `job`, `class`.

The target attribute **class** indicates whether a case is labelled *good* or *bad*.

2.2 Removing the ID field

The first major adjustment I made was to delete the `case_no` attribute. This field is simply an index from 1 to 1000 and does not contain any real information about the applicants. When I initially ran J48, the tree started splitting on rules like “**case_no** ≤ 705”, which clearly suggested that the model was treating the ID as if it were meaningful.

To prevent this kind of artificial pattern and potential overfitting, I removed `case_no` in Weka using the **Remove** filter. Every dataset used in CW2 is built only from the nine descriptive attributes plus the `class` label.

2.3 Three datasets for three tasks

To align with the coursework brief, I generated three ARFF files, with each one set up for a different category of algorithm.

2.3.1 Mixed dataset for classification – 40735842_mixed_CW2.arff

For the J48 model, I largely kept the attribute types unchanged:

- `checking_status`, `credit_history`, `purpose`, `saving_status`, `personal_status`, `job`, and `class` remain nominal.
- `credit_amount` and `age` are treated as numeric.
- `class` is defined as the class attribute.

This configuration is stored as `40735842_mixed_CW2.arff`, which has 9 input attributes plus the class label. A screenshot from Weka’s Preprocess tab, confirming the class setting, is provided in **Appendix A (Fig A1)**.

2.3.2 Fully nominal dataset for Apriori – 40735842_nominal_CW2.arff

Since Apriori requires all attributes to be nominal, I transformed the numeric attributes `credit_amount` and `age` using Weka’s **Discretize** filter:

- Filter: unsupervised → attribute → Discretize
- Attributes: `credit_amount` and `age`
- Bins: 10 (default)

After this step, `credit_amount` and `age` were represented as interval-based categories, for example `(-inf-2067.4]` and `(2067.4-3884.8]`. I then saved this fully nominal version of the data as `german_nominal.arff`.

- **Appendix B**, Fig B1: screenshot with `credit_amount` highlighted, showing its range bins.

- **Appendix B**, Fig B2: screenshot with `age` highlighted, showing its bins.

2.3.3 Numeric-simple dataset for clustering – 40735842_numeric_CW2.arff

For the clustering part, I aimed for a very straightforward setup: focus only on loan amount and age, and then use the class label later to judge the results. To do this, I created a trimmed-down dataset:

- Kept: `credit_amount` (numeric), `age` (numeric), `class` (nominal).
- Removed: every other attribute.

I saved this version as `40735842_numeric_CW2`. During clustering, `credit_amount` and `age` are the variables that actually define the clusters, and `class` is used only in the

“classes to clusters evaluation” to see how good and bad borrowers are spread across those clusters.

- **Appendix C, Fig C1:** screenshot of `40735842_numeric_CW2.arff` in Weka, showing only these three attributes.

3. Data mining experiments

3.1 Classification – J48 decision tree

3.1.1 Setup

- Dataset used: `40735842_mixed_CW2.arff`
- Target variable: class with values good/bad
- Learning algorithm: J48 decision tree (C4.5)
- Parameters: default configuration (confidence factor 0.25, minNumObj = 2)
- Evaluation method: 10-fold cross-validation over the full set of 1,000 cases

3.1.2 Performance

From Weka’s results:

- Overall **accuracy**: roughly 73% (730 out of 1,000 cases classified correctly)
- Confusion matrix:

Actual \ Predicted	good	bad
good (700)	618	82
bad (300)	188	112

This means:

- The model correctly recognises about 88% of good customers.
- It only catches around 37% of bad customers as bad.

In other words, the decision tree is useful and interpretable, and it clearly improves on random guessing, but it still struggles to identify all risky customers.

- **Appendix D, Fig D1:** screenshot showing the pruned J48 tree.
- **Appendix D, Fig D2:** screenshot showing the J48 summary and confusion matrix.

3.1.3 Six rules from the J48 tree

These are the **six main rules** extracted from the J48 tree, written in clean IF–THEN form so they can be dropped straight into the report.

1. Rule J1 – Critical credit history with negative balance
 - o IF `checking_status = "<0"`
 - o AND `credit_history = "critical/other existing credit"`
 - o THEN `class = good`
 - o (covers roughly 67 customers)
2. Rule J2 – Negative balance, existing paid credit, no savings, skilled job
 - o IF `checking_status = "<0"`

- o AND credit_history = "existing paid"
 - o AND saving_status = "no known savings"
 - o AND job = "skilled"
 - o THEN class = bad
3. Rule J3 – Negative balance, existing paid credit, no savings, unskilled resident
 - o IF checking_status = "<0"
 - o AND credit_history = "existing paid"
 - o AND saving_status = "no known savings"
 - o AND job = "unskilled resident"
 - o THEN class = good
 4. Rule J4 – Low savings, radio/TV loan, skilled job
 - o IF checking_status = "<0"
 - o AND credit_history = "existing paid"
 - o AND saving_status = "<100"
 - o AND purpose = "radio/tv"
 - o AND job = "skilled"
 - o THEN class = bad
 5. Rule J5 – Low savings, small radio/TV loan, unskilled resident
 - o IF checking_status = "<0"
 - o AND credit_history = "existing paid"
 - o AND saving_status = "<100"
 - o AND purpose = "radio/tv"
 - o AND job = "unskilled resident"
 - o AND credit_amount ≤ 2100
 - o THEN class = bad
 6. Rule J6 – Low savings, larger radio/TV loan, unskilled resident
 - o IF checking_status = "<0"
 - o AND credit_history = "existing paid"
 - o AND saving_status = "<100"
 - o AND purpose = "radio/tv"
 - o AND job = "unskilled resident"
 - o AND credit_amount > 2100
 - o THEN class = good

3.1.4 What these rules say

The J48 rules make it clear that credit risk is shaped by how attributes interact, not by any single variable on its own:

- A negative checking balance (<0) does not automatically mean “**bad risk**”. In Rules J1 and J3, customers with a negative balance are still treated as good, depending on their credit history and whether they are skilled or unskilled.
- The combination of no savings and a skilled job (Rule J2) comes out as risky. This goes against the intuition that a skilled job is always positive; when paired with a negative balance and no savings, it becomes a red flag.
- For radio/TV loans with low savings (Rules J4–J6), the outcome depends on both loan size and job type, which shows that the tree is capturing relatively fine-grained interactions.

Overall, J48 provides a set of rules that are understandable and usable for a credit officer, even if the model does not successfully flag every bad customer.

3.2 Association rules – Apriori

3.2.1 Setup

- Dataset: 40735842_nominal_CW2.arff

- Algorithm: Apriori

- Main settings in Weka:

- o numRules = 20

- o minMetric (confidence) = 0.7

- o lowerBoundMinSupport ≈ 0.15 in the final run

Apriori was executed in Weka's Associate tab using the complete nominal dataset, including the class attribute.

- **Appendix E, Fig E1:** screenshot showing the “Best rules found” section of the Apriori output.

3.2.2 Six rules from Apriori

The Apriori run produces several high-confidence rules, and the strongest ones all describe patterns of good customers. Here are six of them, written in plain English; in the actual write-up I would insert the exact support and confidence values from `Apriori_output.txt`:

1. Rule A1 – No checking account with a small loan
 - o IF checking_status = "no checking"
 - o AND credit_amount falls in the lowest interval (e.g. (-inf-2067.4])
 - o THEN class = good
 - o (support ≈ 0.16 , confidence ≈ 0.92)
2. Rule A2 – No checking account and a skilled worker
 - o IF checking_status = "no checking"
 - o AND job = "skilled"
 - o THEN class = good
 - o (support ≈ 0.27 , confidence ≈ 0.90)
3. Rule A3 – No checking account and male, single
 - o IF checking_status = "no checking"
 - o AND personal_status = "male single"
 - o THEN class = good
 - o (support ≈ 0.23 , confidence ≈ 0.90)
4. Rule A4 – No checking account alone
 - o IF checking_status = "no checking"
 - o THEN class = good
 - o (support ≈ 0.39 , confidence ≈ 0.88)
5. Rule A5 – No checking account with existing paid credit
 - o IF checking_status = "no checking"
 - o AND credit_history = "existing paid"
 - o THEN class = good
 - o (support ≈ 0.19 , confidence ≈ 0.88)
6. Rule A6 – No checking account and low savings
 - o IF checking_status = "no checking"
 - o AND saving_status = "<100"

- o THEN `class = good`
- o (support ≈ 0.19 , confidence ≈ 0.86)

3.2.3 What these rules say

Taken together, the Apriori rules are quite striking: in this dataset, “no checking account” is not a warning sign. Customers with `checking_status = "no checking"` often appear as good credit risks, particularly when they also:

- work in a skilled job,
- have an existing paid credit history, or
- are applying for a smaller loan.

The high confidence levels (around 0.9 or higher in some cases) indicate that, when the rule conditions are met, the outcome is almost always *good*.

This kind of pattern could push a bank to rethink some of its screening rules. It does not imply that everyone without a checking account should be accepted, but it does suggest that this group is worth evaluating more carefully instead of being treated as automatically risky. Apriori is effective here because it produces simple, quantitative rules—with support and confidence that are easy to communicate to colleagues who are not technical.

3.3 Clustering – SimpleKMeans

3.3.1 Setup

- Dataset: `40735842_numeric_CW2.arff`
- Attributes in the clustering step: `credit_amount` (numeric), `age` (numeric)
- Class attribute used for evaluation only: `class`
- Algorithm: SimpleKMeans
- **Settings in Weka:**
 - o `numClusters = 3`
 - o Euclidean: Distance
 - o “Classes to clusters evaluation” turned on, with the nominal class attribute chosen
- **Appendix F, Fig F1:** screenshot displaying the cluster centroids and the classes-to-clusters evaluation output.

3.3.2 Cluster summary

From the SimpleKMeans results in Weka:

- **Cluster 0 – 140 cases (about 14%)**
 - o Average `credit_amount` ≈ 8893.8
 - o Average `age` ≈ 35.1
 - o 84 good vs 56 bad $\rightarrow$ roughly 60% good, 40% bad
- **Cluster 1 – 239 cases (about 24%)**
 - o Average `credit_amount` ≈ 2522.4
 - o Average `age` ≈ 51.5
 - o 182 good vs 57 bad $\rightarrow$ roughly 76% good, 24% bad

- **Cluster 2 – 621 cases (about 62%)**

- o Average `credit_amount` ≈ 2307.7
- o Average `age` ≈ 29.5
- o 434 good vs 187 bad $\rightarrow$ roughly 70% good, 30% bad

When Weka matches clusters to the class labels, it reports about 50.9% of instances as “**incorrectly clustered**”. This is acceptable, because KMeans is unsupervised: it is grouping cases by loan amount and age, not actively trying to reproduce the good/bad classification.

3.3.3 Six patterns from the clusters

Based on the cluster statistics, we can describe six clear patterns:

- **C1 – Very large loans carry extra risk**
 - o Cluster 0 has the highest mean loan ($\sim 8,900$) and only about 60% good customers.
 - o In other words: when someone in their mid-30s takes a very large loan, the probability of a bad outcome rises noticeably.
- **C2 – Older, moderate-loan customers look safest**
 - o Cluster 1 is made up of older applicants (around 51 years) with mid-sized loans (around 2,500).
 - o Roughly 76% of them are good, so this is the safest group in the clustering.
- **C3 – Younger, moderate-loan customers are in the middle**
 - o Cluster 2 contains younger applicants (around 30 years) with similar moderate loans ($\sim 2,300$).
 - o About 70% are good – better than the high-loan cluster, but clearly weaker than Cluster 1.
- **C4 – Loan size has a stronger impact than age**
 - o Comparing Cluster 0 with Cluster 2: the ages are fairly close, but the huge loans in Cluster 0 push the bad rate up to 40%.
 - o This points to loan amount as a key factor in credit risk.
- **C5 – Age helps when loan amounts are sensible**
 - o When we compare Cluster 1 (older borrowers, moderate loans) with Cluster 2 (younger borrowers, similar loans), the older group shows higher repayment reliability for roughly the same loan size.
- **C6 – Clusters form a practical segmentation for decision-making**
 - o The three clusters give a simple segmentation scheme:
 - Cluster 0: small, high-risk “large-loan” group.
 - Cluster 1: medium-sized, low-risk “older moderate-loan” group.
 - Cluster 2: large, medium-risk “younger moderate-loan” group.
 - o A bank could use this to tailor its policies, for example by tightening criteria or interest rates for Cluster 0 while being more flexible with Cluster 1.

4. Overall findings and comparison

Bringing the three methods together:

- **J48** gave a decision tree with about 73% accuracy and a set of rules that are easy to read. The tree highlights that risk depends on combinations of checking status, credit history,

savings, purpose and job. Some profiles that sound reassuring (such as “skilled worker”) become risky once they are combined with a negative balance and no savings.

- **Apriori** produced a list of strong association rules, almost all of which point towards good customers, particularly when `checking_status = "no checking"`. This is an interesting and slightly counter-intuitive pattern, because it goes against the simple rule “**no current account = bad risk**”. It shows how association rules can uncover relationships that are very easy to explain to non-technical staff.

- **SimpleKMeans** on `credit_amount` and `age` divided the data into three clear clusters:

- a small group taking very large loans (highest risk),
- older applicants with moderate loans (safest group), and
- younger applicants with moderate loans (medium risk).

KMeans is not trying to match the good/bad labels, but it still offers a natural way to segment customers.

For **practical decision support**, I would choose **J48**:

- it directly links applicant features to a prediction of good or bad, and
- the resulting rules can be reviewed in terms of fairness and business policy.

For **explaining patterns** to colleagues who are not familiar with data mining, **Apriori** is very effective because its “IF ... THEN ...” rules, together with support and confidence, read almost like plain policy statements.

In contrast, **clustering** is better seen as a tool for exploration: it helps to map out the overall customer base and highlight segments that might need different treatment, rather than providing a yes/no decision rule on its own.

5. References

- Witten, I. H., Frank, E., Hall, M. A. and Pal, C. J. (2017) Data Mining: Practical Machine Learning Tools and Techniques. 4th edn. Morgan Kaufmann.
- Han, J., Kamber, M. and Pei, J. (2012) Data Mining: Concepts and Techniques. 3rd edn. Morgan Kaufmann.
- SET09120 Lecture Notes and Lab Materials, Edinburgh Napier University (2025).

6. Appendix

- **Appendix A – Mixed dataset for J48**
 - **Fig A1:** Preprocess view of `40735842_mixed_CW2.arff` (9 attributes, class set as class).

Weka Explorer

PreprocessClassifyClusterAssociateSelect attributesVisualize

Open file...Open URL...Open DB...Generate...UndoEdit...Save...

Filter

ChooseNoneApplyStop

Current relation

Relation: SET09120_CW1_40735842_CLEANING

Instances: 1000

Attributes

Attributes: 10

Sum of weights: 1000

Attributes

AllNoneInvertPattern

No.	Name
1	☐ case_no
2	☐ checking_status
3	☐ credit_history
4	☐ purpose
5	☐ credit_amount
6	☐ saving_status
7	☐ personal_status
8	☐ age
9	☐ job
10	☒ class

Remove

Selected attribute

Name: class

Missing: 0 (0%)

Distinct: 2

Type: Nominal

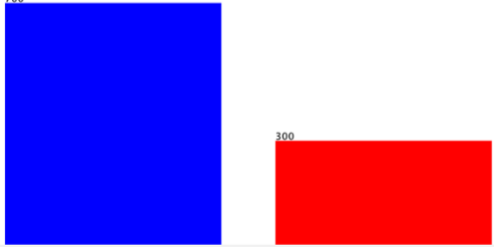
Unique: 0 (0%)

No.	Label	Count	Weight
1	good	700	700
2	bad	300	300

Class: class (Nom)Visualize All

700

300

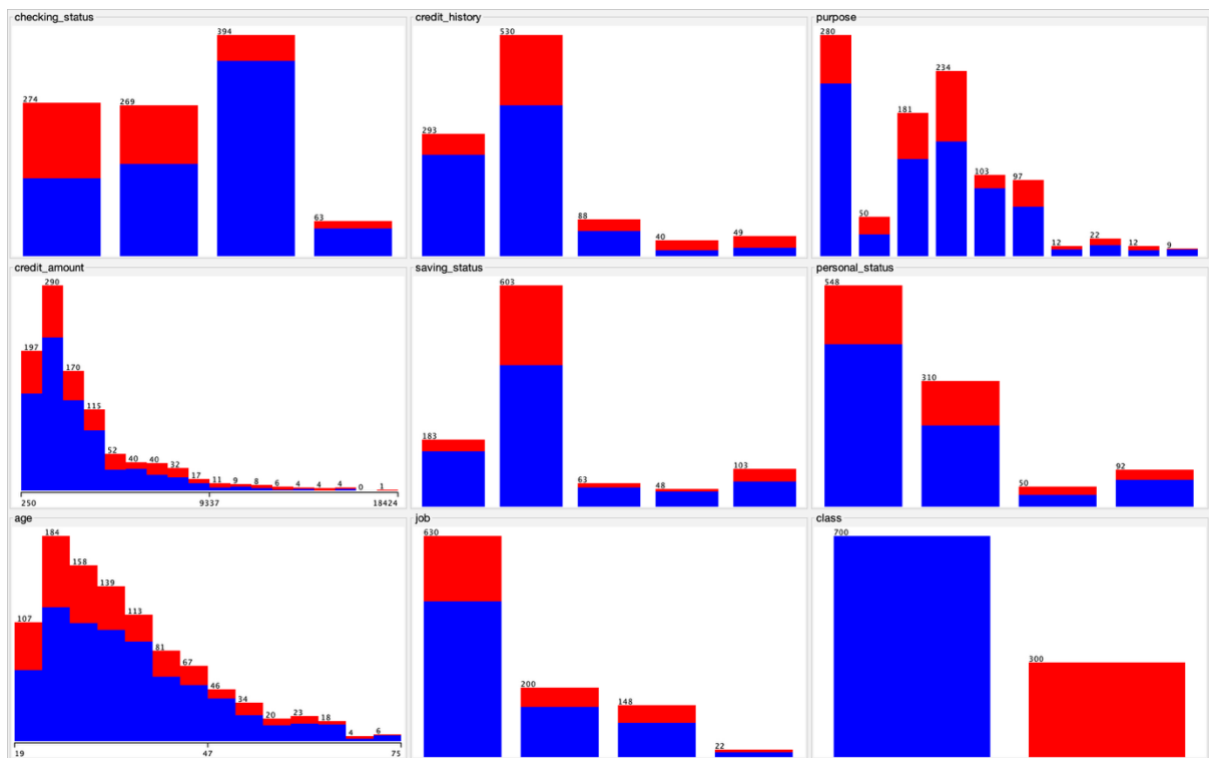
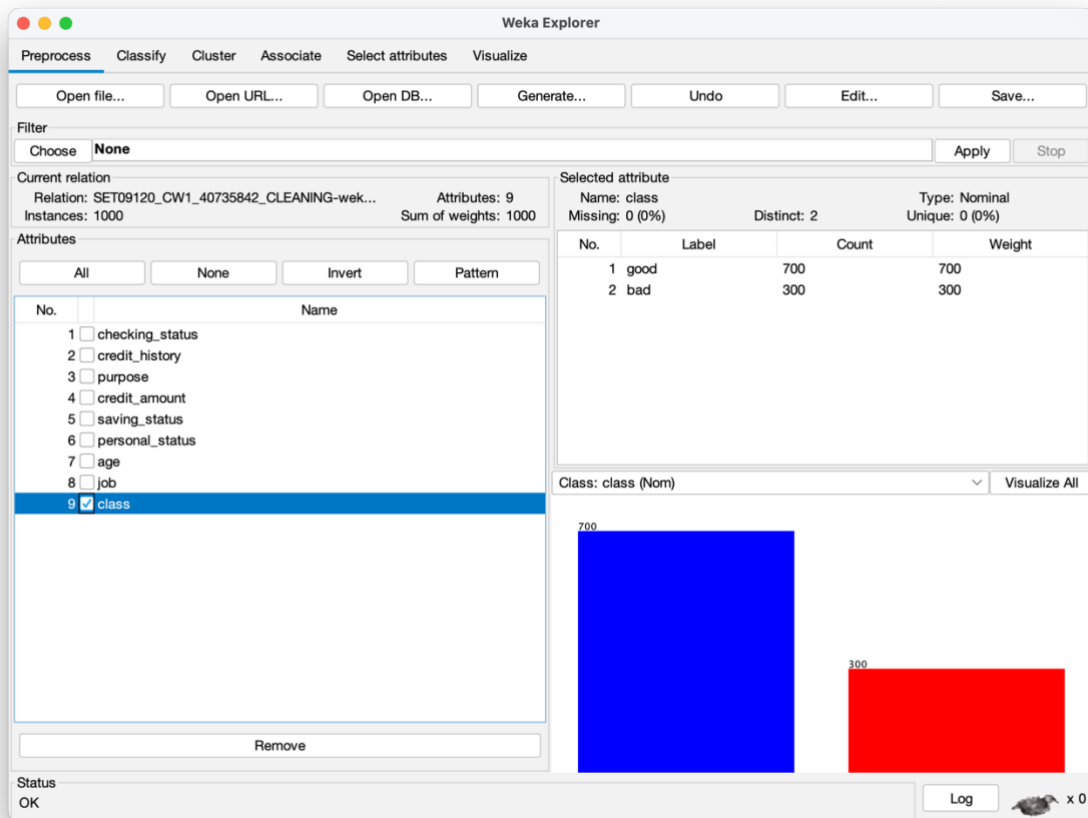


Status

OK

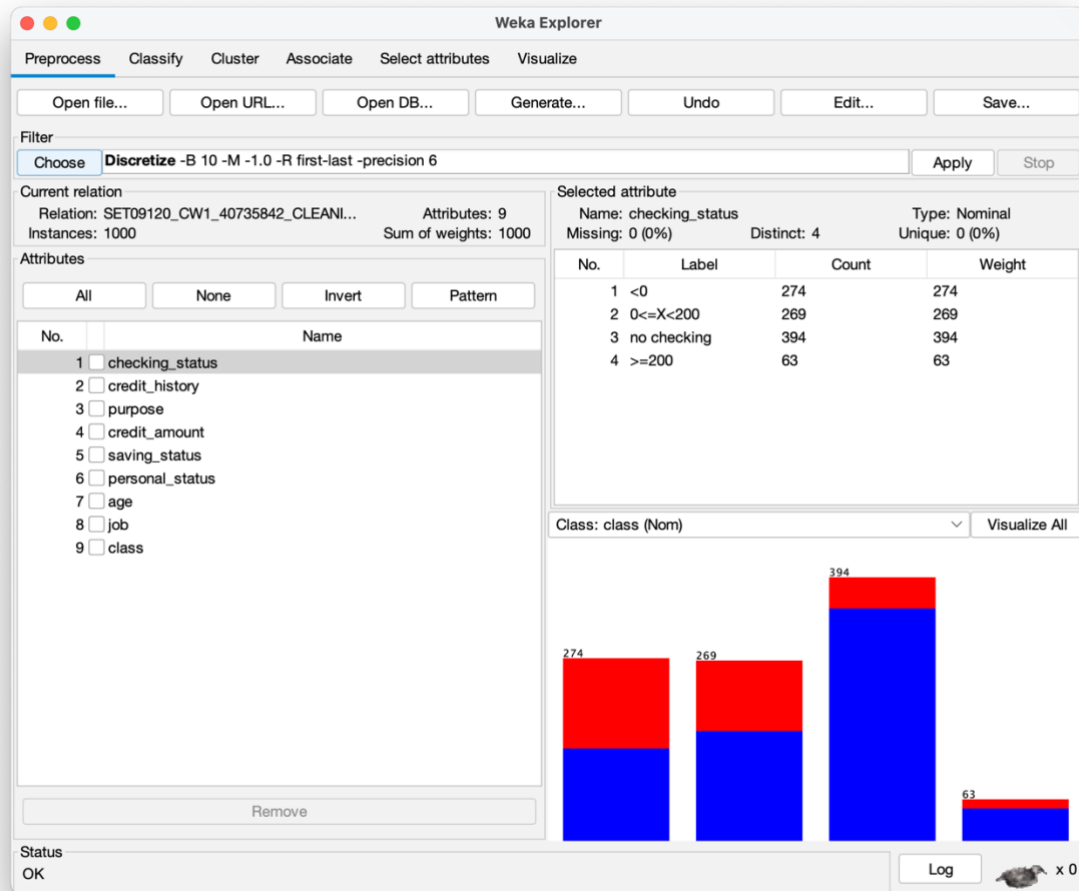
Log

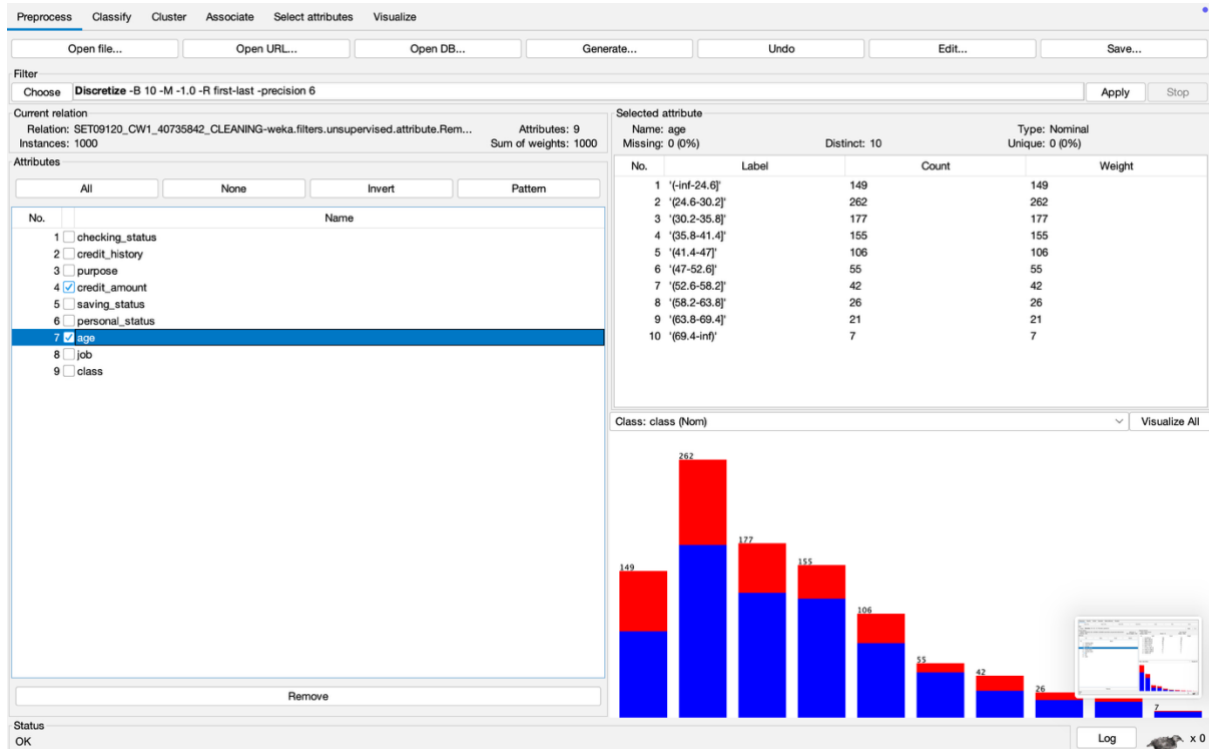
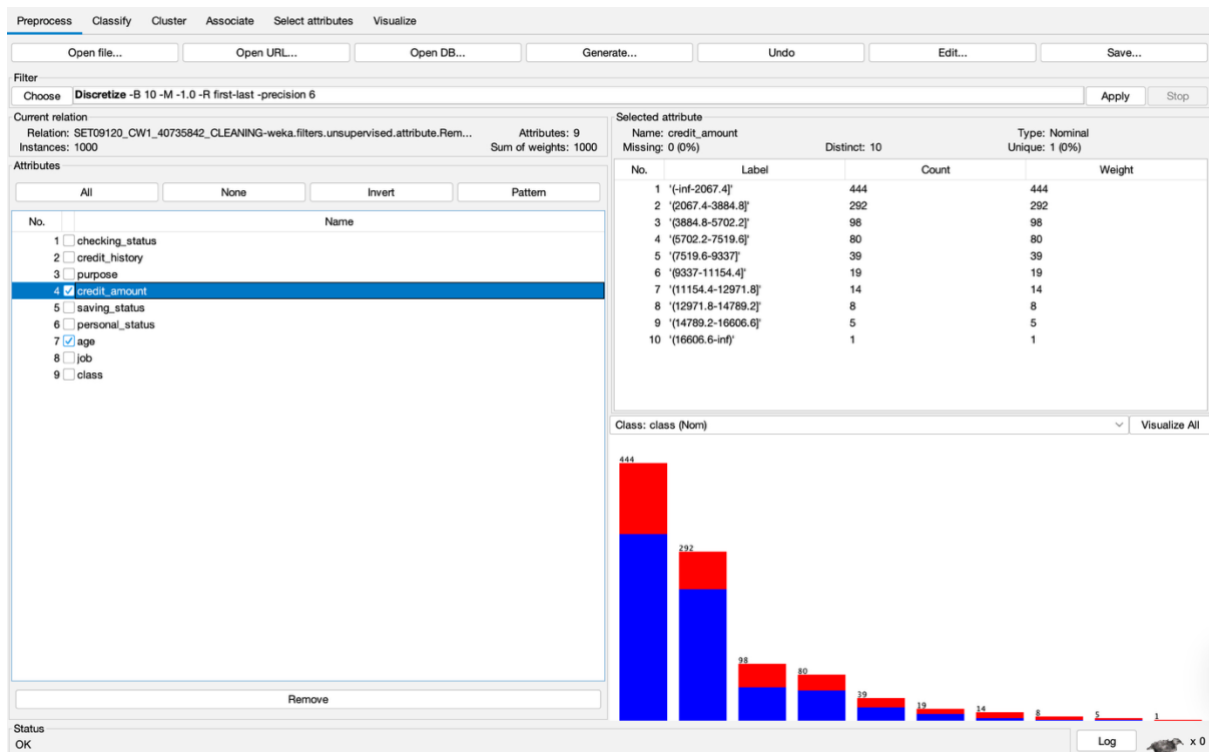
 x 0



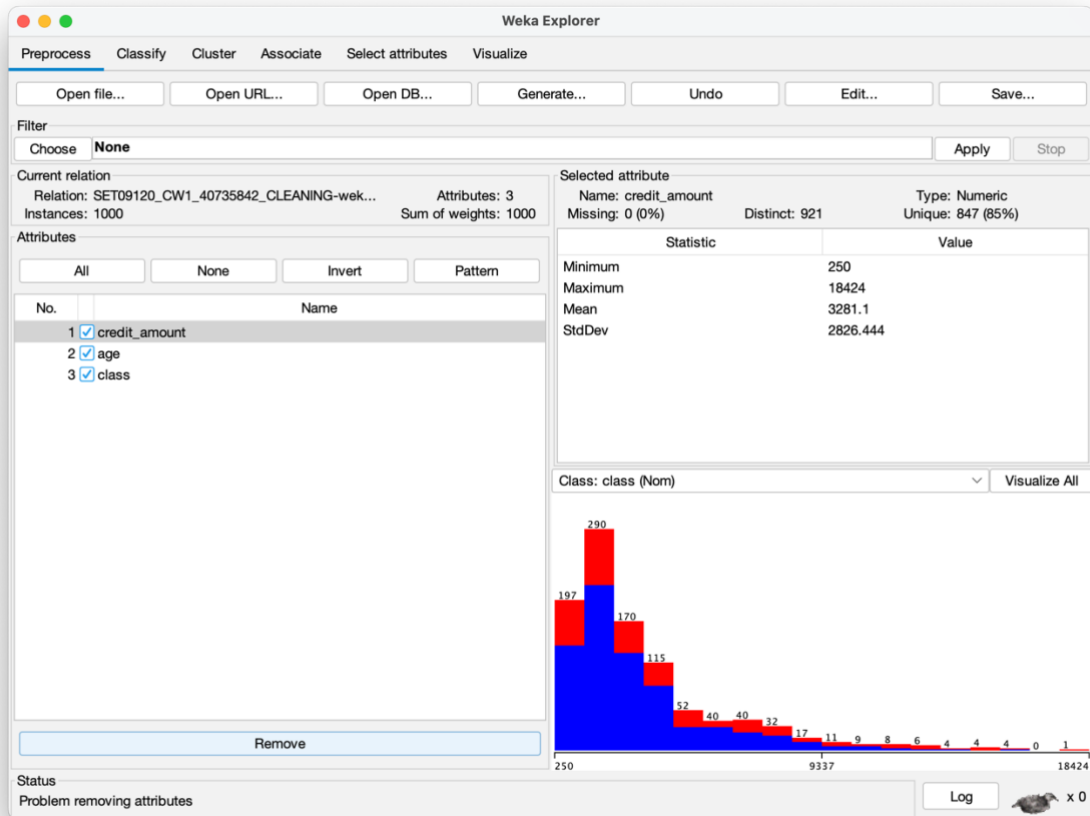
- **Appendix B – Discretised dataset for Apriori**
 - **Fig B1:** `credit_amount` selected, showing discretised nominal ranges.

- **Fig B2:** age selected, showing discretised nominal ranges.

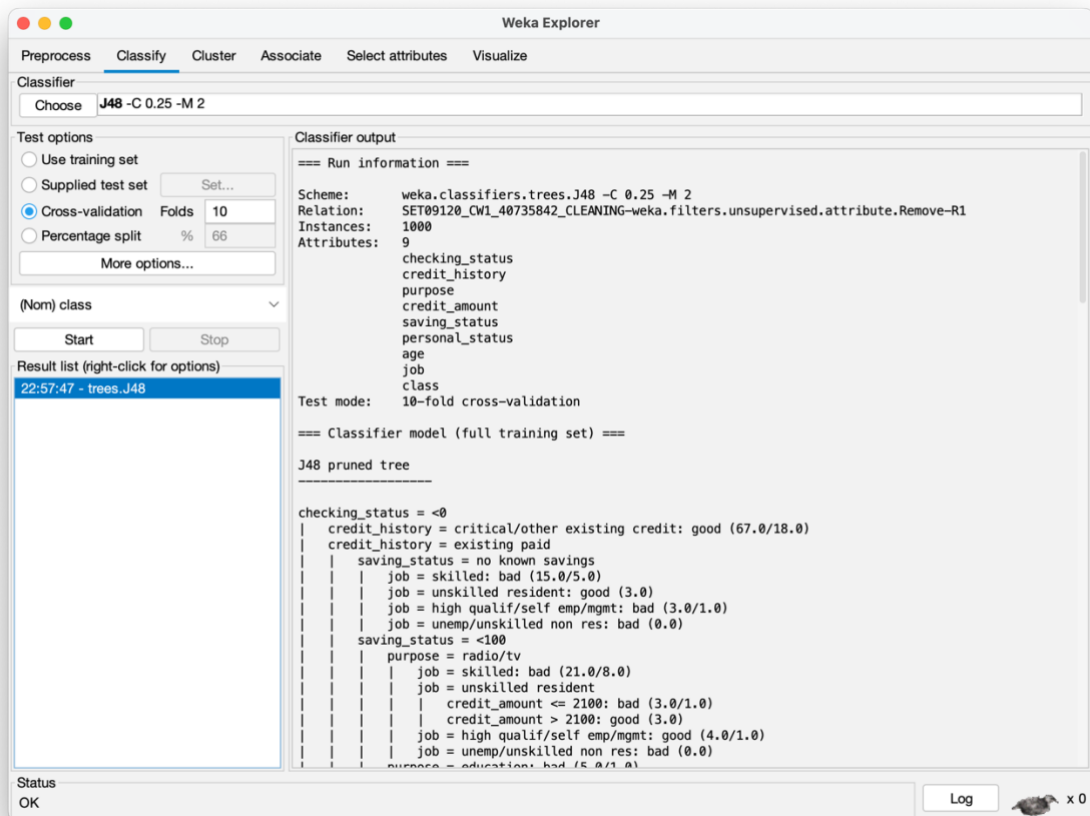


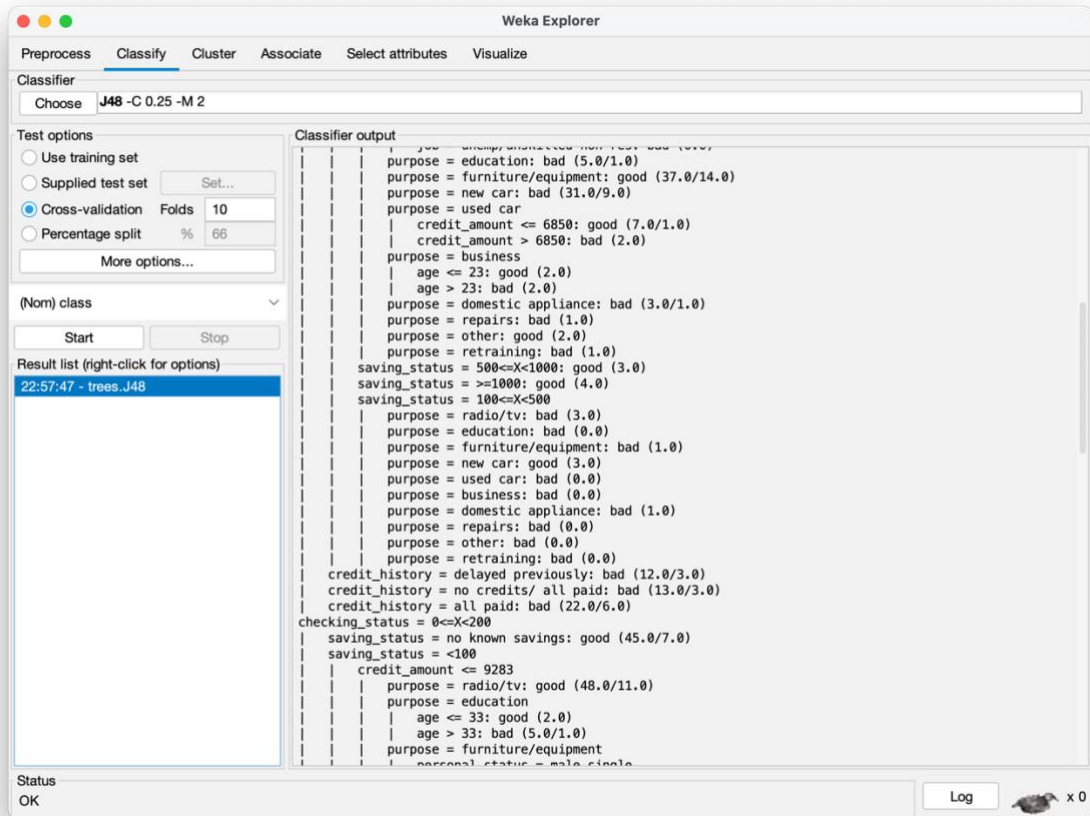


- **Appendix C – Numeric-simple dataset for clustering**
 - **Fig C1:** 40735842_numeric_CW2.arff with only credit_amount, age, class.



- **Appendix D – J48 outputs**
 - **Fig D1:** J48 pruned tree (the textual tree screenshot).
 - **Fig D2:** J48 summary: accuracy, confusion matrix, stats.





Preprocess

Classify

Cluster

Associate

Select attributes

Visualize

Classifier

Choose **J48 -C 0.25 -M 2**

Test options

☐ Use training set

☐ Supplied test set

Set...

☒ Cross-validation

Folds **10**

☐ Percentage split

% **66**

More options...

(Nom) class

Start Stop

Result list (right-click for options)

22:57:47 - trees.J48

Classifier output

```

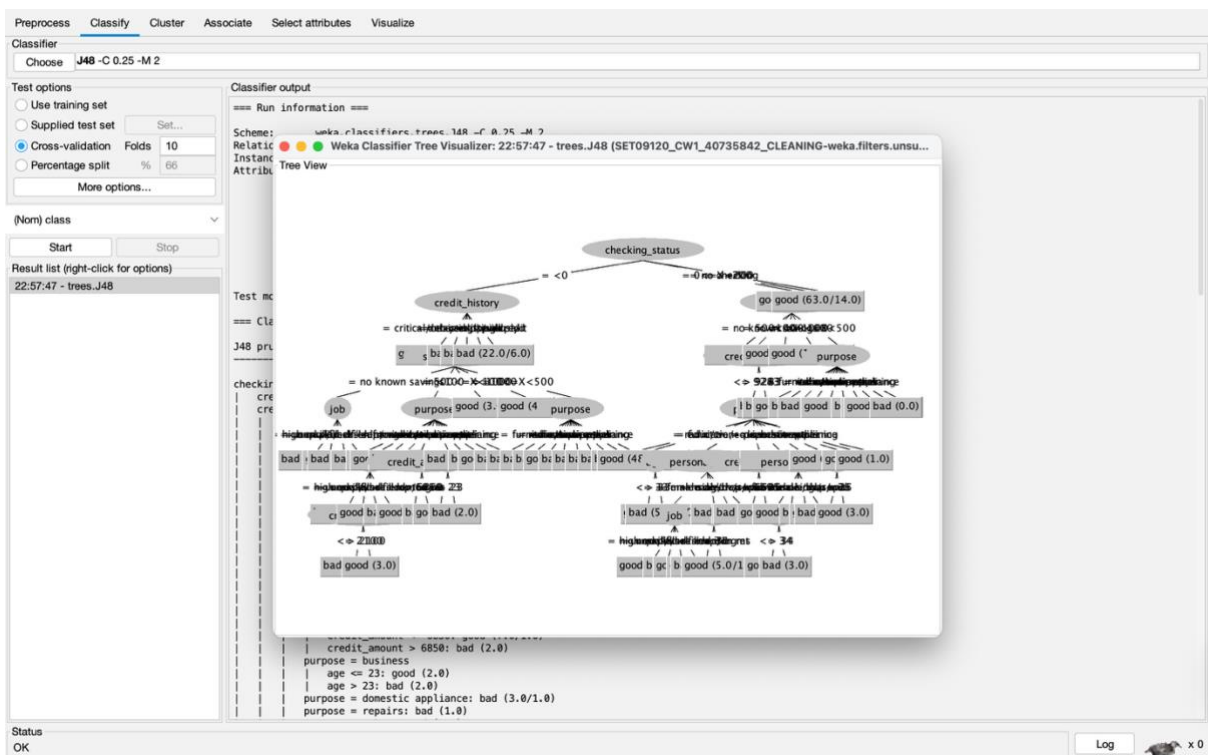
| | | purpose = furniture/equipment
| | |   personal_status = male single
| | |     | job = skilled: good (4.0/1.0)
| | |     | job = unskilled resident: bad (2.0)
| | |     | job = high qualif/self emp/mgmt: good (2.0)
| | |     | job = unemp/unskilled non res: good (0.0)
| | |   personal_status = female div/dep/mar
| | |     | age <= 31: bad (7.0/1.0)
| | |     | age > 31: good (5.0/1.0)
| | |   personal_status = male div/sep: bad (3.0/1.0)
| | |   personal_status = male mar/wid: good (1.0)
| | | purpose = new car
| | |   credit_amount <= 5595: bad (23.0/7.0)
| | |   credit_amount > 5595: good (4.0)
| | | purpose = used car: good (8.0/1.0)
| | | purpose = business
| | |   personal_status = male single
| | |     | age <= 34: good (2.0)
| | |     | age > 34: bad (3.0)
| | |   personal_status = female div/dep/mar: good (6.0/3.0)
| | |   personal_status = male div/sep: bad (2.0)
| | |   personal_status = male mar/wid: good (3.0)
| | | purpose = domestic appliance: good (1.0)
| | | purpose = repairs
| | |   age <= 35: bad (3.0/1.0)
| | |   age > 35: good (3.0)
| | | purpose = other: good (1.0)
| | | purpose = retraining: good (1.0)
| | |   credit_amount > 9283: bad (13.0)
| | | saving_status = 500<=X<1000: good (11.0/3.0)
| | | saving_status = >=1000: good (14.0/4.0)
| | | saving_status = 100<=X<500
| | |   | purpose = radio/tv: bad (8.0/2.0)
| | |   | purpose = education: good (2.0/1.0)
| | |   | purpose = furniture/equipment: bad (4.0/1.0)
| | |   | purpose = new car: bad (15.0/5.0)
| | |   | purpose = used car: good (3.0)
| | |   | purpose = business: good (11.0/3.0)
| | |   | purpose = domestic appliance: bad (0.0)

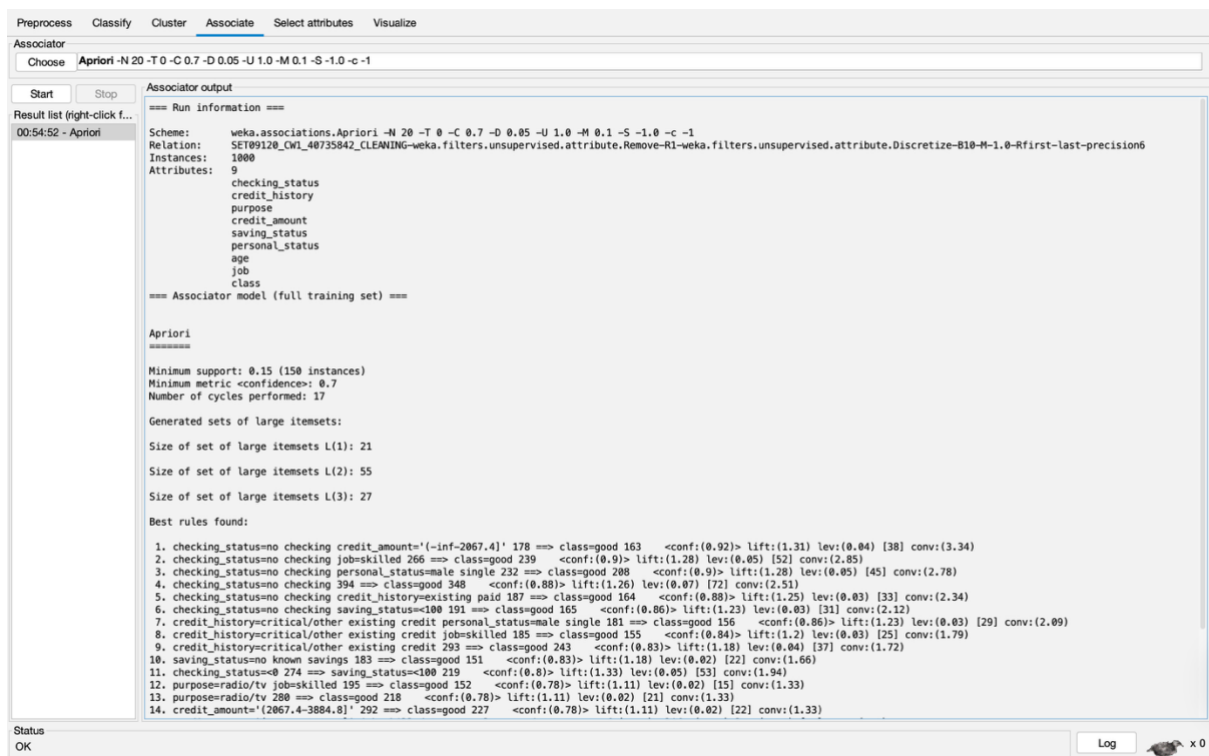
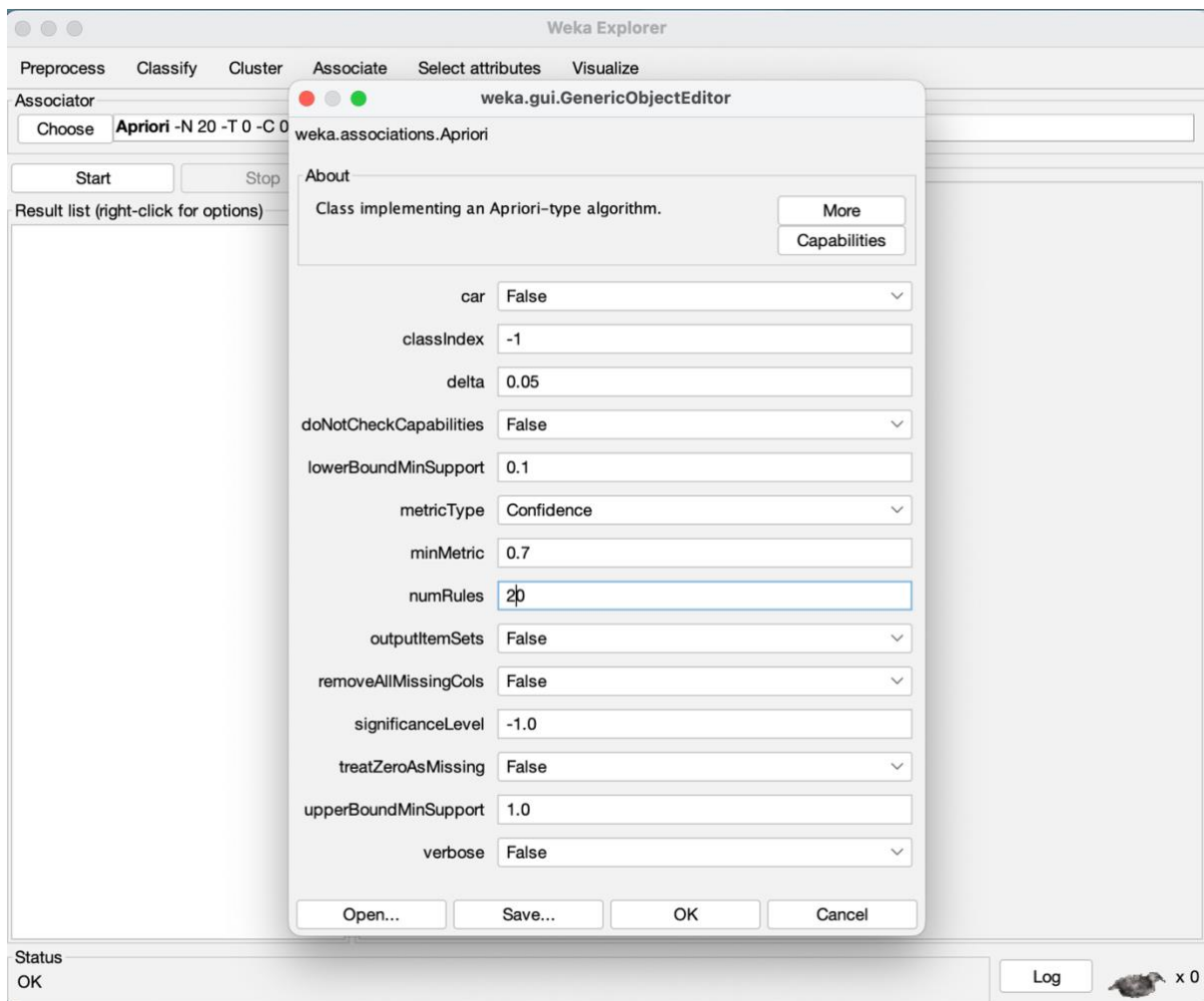
```

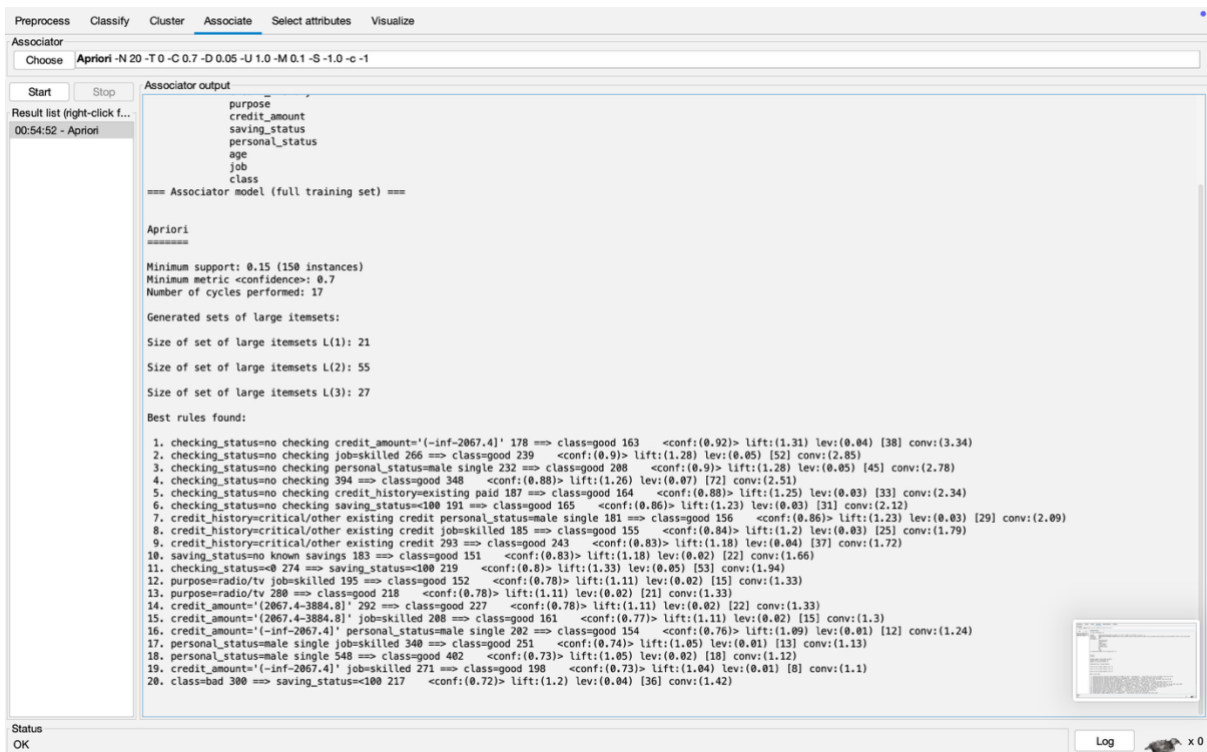
Status

OK

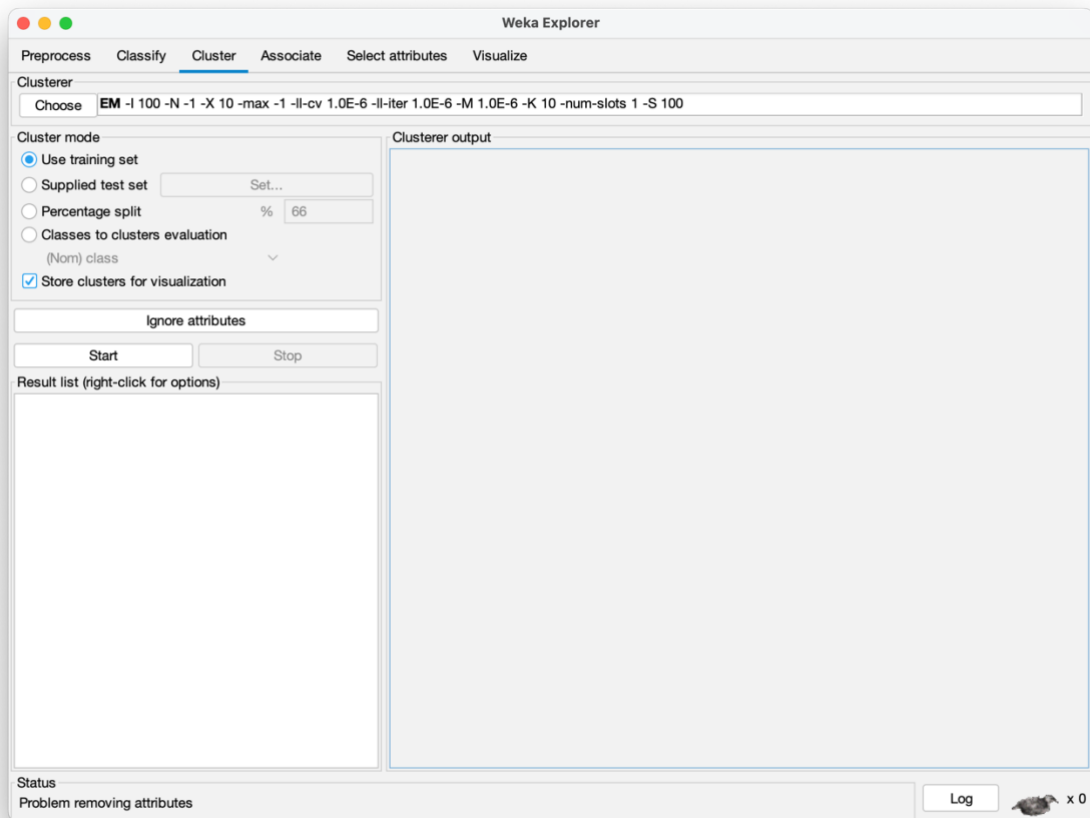
Log x 0

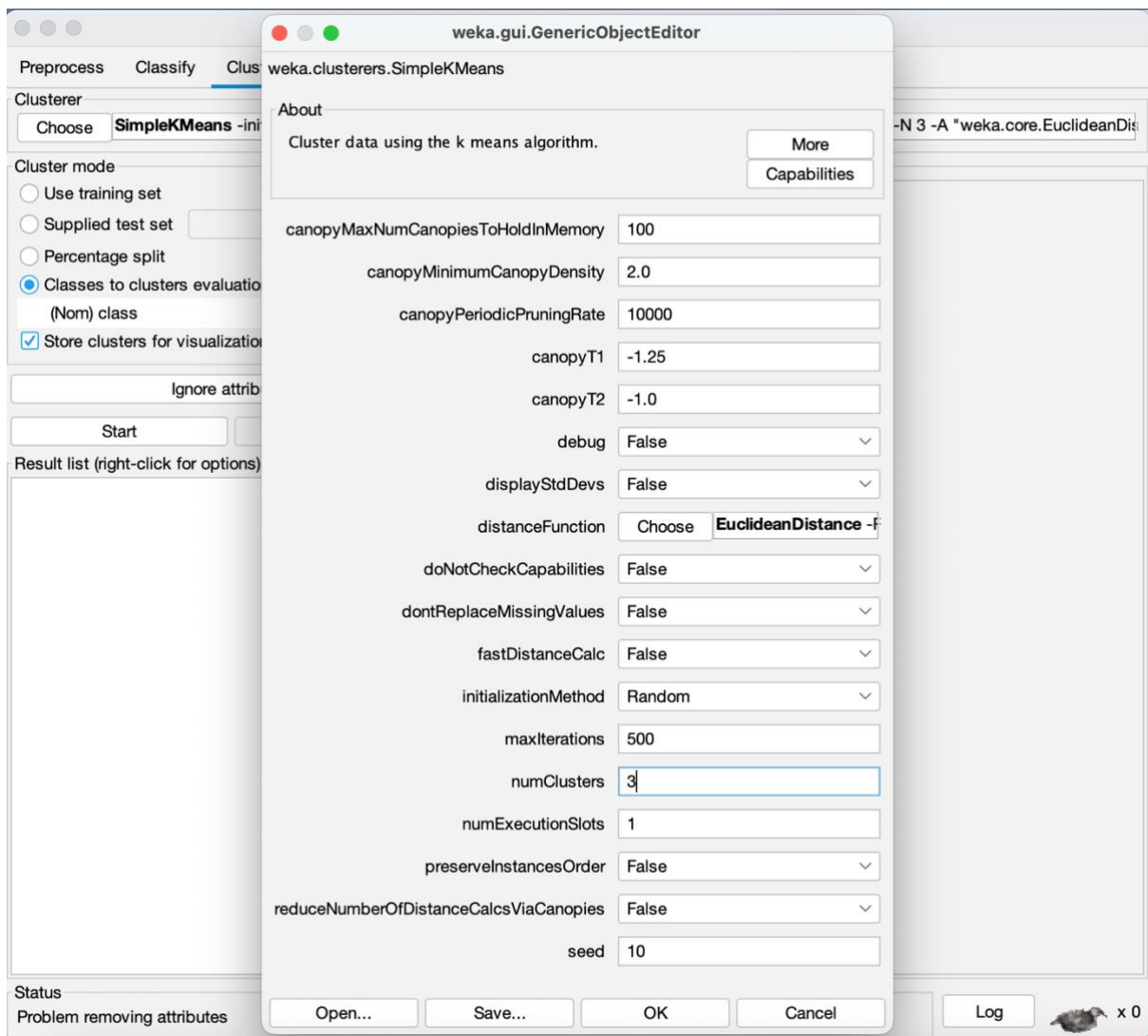






- **Appendix F – KMeans outputs**
 - **Fig F1:** SimpleKMeans output with final cluster centroids + classes-to-clusters evaluation.





Weka Explorer

PreprocessClassifyClusterAssociateSelect attributesVisualize

Clusterer

ChooseSimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 3 -A "weka.core.EuclideanDis

Cluster mode

☐ Use training set

☐ Supplied test set

Set...

☐ Percentage split

%66

☒ Classes to clusters evaluation

(Nom) class

☒ Store clusters for visualization

Ignore attributes

Start

Stop

Result list (right-click for options)

00:11:20 - SimpleKMeans

Clusterer output

Cluster 2: 1155,40

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute	Full Data	Cluster#	0	1	2
	(1000.0)	(140.0)	(239.0)	(621.0)	
credit_amount	3281.1	8893.7786	2522.4393	2307.7424	
age	35.54	35.1	51.5063	29.4944	

Time taken to build model (full training data) : 0.04 seconds

=== Model and evaluation on training set ===

Clustered Instances

0	140	(14%)
1	239	(24%)
2	621	(62%)

Class attribute: class

Classes to Clusters:

0	1	2	<-- assigned to cluster
84	182	434	good
56	57	187	bad

Cluster 0 <-- No class

Cluster 1 <-- bad

Cluster 2 <-- good

Incorrectly clustered instances :

509.0	50.9	%
-------	------	---

Status

OK

Log

x 0